

Competitive Programming

Section 41 Editorial: Euler Tour, LCA, and Binary Lifting

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. Tree Distances	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	5
2.7	Tests that matter	5
3	B. Max Edge On Path	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	7
3.7	Tests that matter	7
4	C. Kth Node On Path	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	9
4.7	Tests that matter	9
5	What to carry forward	10

1 Editorial overview

1.1 Detailed solution guide

A. Tree Distances. Root the tree once and compute $\text{depth}[v]$ and $\text{up}[k][v]$, the 2^k -th ancestor of v . To answer $\text{lca}(u, v)$, first lift the deeper vertex until both depths match. Then lift both vertices from high powers down to low powers whenever their ancestors differ. Their parents are then the LCA. The distance formula is $\text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[\text{lca}]$.

B. Maximum Edge on a Path. Store one more table: $\text{mx}[k][v]$, the maximum edge weight on the path from v up by 2^k edges. When lifting a vertex, merge the corresponding mx value into the answer. The path from u to v splits into $u \rightarrow \text{lca}$ and $v \rightarrow \text{lca}$, so lifting both sides and taking the maximum over the chunks covers every path edge exactly once.

C. k-th Node on a Path. Let $w = \text{lca}(u, v)$. The path length in vertices is $\text{depth}[u] - \text{depth}[w] + 1 + \text{depth}[v] - \text{depth}[w]$. If k lies on the u to w part, jump $k-1$ steps upward from u . Otherwise the answer lies on the w to v part; convert it to a jump upward from v by counting how many vertices remain after position k .

Common pitfalls. Binary lifting uses powers of two, so allocate enough levels for the maximum n . For C, be explicit about whether k is 1-based; the local statement uses 1-based path positions.

This section has one preprocessing object: a binary-lifting table rooted at node 1. The three exercises reuse it with different query payloads.

- A uses only depth and up to compute $\text{lca}(u, v)$, then applies the distance formula.
- B stores $\text{mx}[k][v]$, the maximum edge on the upward jump of length 2^k . Every time the query lifts a vertex, it merges the corresponding maximum.
- C first computes the LCA and the total path length in nodes. If the requested k stays on the u side, jump upward from u ; otherwise convert it to a jump upward from v .

The proof pattern is the same in all three: the path is decomposed into disjoint upward power-of-two chunks, and the table stores the exact ancestor or aggregate for each such chunk.

2 A. Tree Distances

2.1 Restatement

Given a rooted tree and queries u, v , print the number of edges on the path.

2.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

2.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

2.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);
    int n,q;if(!(cin>>n>>q)) return 0; int LOG=1; while((1<<LOG)<=n+1) LOG++;
    vector<vector<int>> up(LOG, vector<int>(n+1)); vector<int> depth(n+1);
    for(int v=2;v<=n;v++){cin>>up[0][v]; depth[v]=depth[up[0][v]]+1;}
    for(int k=1;k<LOG;k++) for(int v=1;v<=n;v++) up[k][v]=up[k-1][up[k-1][v]];
    auto lift=[&](int v,int d){for(int k=0;k<LOG;k++) if(d&(1<<k)) v=up[k][v]; return v;};
    auto lca=[&](int a,int b){if(depth[a]<depth[b]) swap(a,b); a=lift(a,depth[a]-depth[b]); if
    while(q--){int a,b;cin>>a>>b; int c=lca(a,b); cout<<depth[a]+depth[b]-2*depth[c]<<'\n';}
}
```

2.6 Full Python solution

```
import sys

def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,q=data[0],data[1]; idx=2
    LOG=max(1,(n+1).bit_length())
    up=[[0]*(n+1) for _ in range(LOG)]; depth=[0]*(n+1)
    for v in range(2,n+1):
        p=data[idx]; idx+=1; up[0][v]=p; depth[v]=depth[p]+1
    for k in range(1,LOG):
        for v in range(1,n+1): up[k][v]=up[k-1][up[k-1][v]]
    def lift(v,d):
        bit=0
        while d:
            if d&1: v=up[bit][v]
            d//=2; bit+=1
        return v
    def lca(a,b):
        if depth[a]<depth[b]: a,b=b,a
        a=lift(a,depth[a]-depth[b])
        if a==b: return a
        for k in range(LOG-1,-1,-1):
            if up[k][a]!=up[k][b]: a=up[k][a]; b=up[k][b]
        return up[0][a]
    out=[]
    for _ in range(q):
        a,b=data[idx],data[idx+1]; idx+=2; c=lca(a,b)
        out.append(str(depth[a]+depth[b]-2*depth[c]))
    print("\n".join(out))
if __name__=="__main__": main()
```

2.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

3 B. Max Edge On Path

3.1 Restatement

Each edge to a parent has a weight. For each $u \ v$, print the maximum edge weight on the path.

3.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

3.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

3.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);
    int n,q;if(!(cin>>n>>q)) return 0; int LOG=1; while((1<<LOG)<=n+1) LOG++;
    vector<vector<int>> up(LOG, vector<int>(n+1)), mx(LOG, vector<int>(n+1)); vector<int> depth
    for(int v=2;v<=n;v++){int p,w;cin>>p>>w; up[0][v]=p; mx[0][v]=w; depth[v]=depth[p]+1;}
    for(int k=1;k<LOG;k++) for(int v=1;v<=n;v++){int mid=up[k-1][v]; up[k][v]=up[k-1][mid]; mx[k]
    auto lift=[&](int &v,int d){int ans=0; for(int k=0;k<LOG;k++) if(d&(1<<k)){ans=max(ans,mx[k]
    while(q--){int a,b;cin>>a>>b; int ans=0; if(depth[a]<depth[b]) swap(a,b); ans=max(ans,lift
    }
```

3.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,q=data[0],data[1]; idx=2; LOG=max(1,(n+1).bit_length())
    up=[[0]*(n+1) for _ in range(LOG)]; mx=[[0]*(n+1) for _ in range(LOG)]; depth=[0]*(n+1)
    for v in range(2,n+1):
        p,w=data[idx],data[idx+1]; idx+=2; up[0][v]=p; mx[0][v]=w; depth[v]=depth[p]+1
    for k in range(1,LOG):
        for v in range(1,n+1):
            mid=up[k-1][v]; up[k][v]=up[k-1][mid]; mx[k][v]=max(mx[k-1][v],mx[k-1][mid])
    def lift(v,d):
        ans=0; bit=0
        while d:
            if d&1: ans=max(ans,mx[bit][v]); v=up[bit][v]
            d//=2; bit+=1
        return v,ans
    out=[]
    for _ in range(q):
        a,b=data[idx],data[idx+1]; idx+=2; ans=0
        if depth[a]<depth[b]: a,b=b,a
        a,val=lift(a,depth[a]-depth[b]); ans=max(ans,val)
        if a!=b:
            for k in range(LOG-1,-1,-1):
                if up[k][a]!=up[k][b]:
                    ans=max(ans,mx[k][a],mx[k][b]); a=up[k][a]; b=up[k][b]
            ans=max(ans,mx[0][a],mx[0][b])
        out.append(str(ans))
    print("\n".join(out))
if __name__=="__main__": main()
```

3.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

4 C. Kth Node On Path

4.1 Restatement

For query $u \ v \ k$, print the k -th node on the path from u to v , or -1 .

4.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

4.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

4.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);
    int n,q;if(!(cin>>n>>q)) return 0; int LOG=1; while((1<<LOG)<=n+1) LOG++;
    vector<vector<int>> up(LOG, vector<int>(n+1)); vector<int> depth(n+1);
    for(int v=2;v<=n;v++){cin>>up[0][v]; depth[v]=depth[up[0][v]]+1;}
    for(int k=1;k<LOG;k++) for(int v=1;v<=n;v++) up[k][v]=up[k-1][up[k-1][v]];
    auto lift=[&](int v,int d){for(int k=0;k<LOG;k++) if(d&(1<<k)) v=up[k][v]; return v;};
    auto lca=[&](int a,int b){if(depth[a]<depth[b]) swap(a,b); a=lift(a,depth[a]-depth[b]); if
    while(q--){int u,v,k;cin>>u>>v>>k; int c=lca(u,v); int left=depth[u]-depth[c]+1,total=depth
    }
```

4.6 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,q=data[0],data[1]; idx=2; LOG=max(1,(n+1).bit_length())
    up=[[0]*(n+1) for _ in range(LOG)]; depth=[0]*(n+1)
    for v in range(2,n+1):
        p=data[idx]; idx+=1; up[0][v]=p; depth[v]=depth[p]+1
    for k in range(1,LOG):
        for v in range(1,n+1): up[k][v]=up[k-1][up[k-1][v]]
    def lift(v,d):
        for k in range(LOG):
            if d>>k & 1: v=up[k][v]
        return v
    def lca(a,b):
        if depth[a]<depth[b]: a,b=b,a
        a=lift(a,depth[a]-depth[b])
        if a==b: return a
        for k in range(LOG-1,-1,-1):
            if up[k][a]!=up[k][b]: a=up[k][a]; b=up[k][b]
        return up[0][a]
    out=[]
    for _ in range(q):
        u,v,k=data[idx],data[idx+1],data[idx+2]; idx+=3; c=lca(u,v); left=depth[u]-depth[c]+1
        if k<1 or k>total: out.append("-1")
        elif k<=left: out.append(str(lift(u,k-1)))
        else: out.append(str(lift(v,total-k)))
    print("\n".join(out))
if __name__=="__main__": main()
```

4.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

5 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.